Лабораторная №2. Backend

Разработать REST API для системы управления студентами общежития.
Допустимые технологии в зависимости от варианта:
Python с использованием FastAPI, Flask, Django REST Framework.

Клиентский интерфейс должен быть изменен согласно следующим требованиям:

- Интерфейс должен поддерживать фильтрацию по свойствам студента
- Интерфейс должен работать в режиме stateless (бизнес-логика должна мигрировать на серверную часть)

API должно работать с той же моделью студента, которая использовалась в Лабораторной работе №1.

Обязательные запросы

GET /api/requests
GET /api/requests/:id
POST /api/requests
PATCH /api/requests/:id
DELETE /api/requests/:id
QUERY /api/requests

Для получения списка необходимо реализовать фильтрацию через query-параметры, например:
`GET /api/requests?group=M3301&dormitory=8`.

Если количество query-параметров становится слишком большим, необходимо использовать метод `QUERY`.

Обязательные требования

1. принимать и возвращать данные в формате `JSON с корректными HTTP-кодами`
2. выполнять дополнительно серверную валидацию входных данных
3. обязательно наличие у студента уникального идентификатора (ИСУ ID)
4. хранить данные в памяти приложения или JSON-файле
5. разделить маршруты, бизнес-логику и работу с данными по отдельным файлам

возвращать ошибки в едином формате, используя соответствующие HTTP-коды: `200`, `201`, `204`, `400`, `404`, `409`, `422`, `500`.

Вопросы к защите лабораторной работы:

1. Python в серверной веб-разработке.
2. Backend-фреймворки Python: FastAPI, Flask, Django REST Framework.
3. Типизация данных в Python и аннотации типов.
4. Модели данных: dataclass, Pydantic-модели, словари и классы.
5. Исключения в Python и их обработка в Backend-приложениях.
6. Декораторы и их применение в веб-фреймворках.
7. Контекст выполнения запроса и область видимости данных.
8. Синхронный и асинхронный код в Python.
9. Итераторы, генераторы и работа с коллекциями данных.
10. Модули, пакеты и разделение Backend-приложения по слоям.
11. REST и RESTful API.
12. HTTP-методы и коды состояния.
13. Маршрутизация, path-параметры и query-параметры.
14. JSON, сериализация и десериализация.
15. Серверная валидация и обработка ошибок.
16. Stateless-архитектура.
17. Хранение и фильтрация данных в памяти и JSON-файлах.
18. CRUD-операции и уникальная идентификация ресурсов.